

Session 6

HTML DOM Navigation

HTML DOM interprets an HTML document as a hierarchical tree of nodes. This session focuses on DOM Navigation, which allows JavaScript to traverse this tree using parent-child-sibling relationships. Additionally, this session covers dynamic DOM Manipulation: creating, inserting, and removing HTML elements programmatically without page reloads.

Objectives

✕ Do not copy. Read for understanding and the viva

- Navigate between DOM nodes using built-in relationship properties
- Dynamically create and attach new HTML elements to the document
- Remove existing elements or child nodes from the DOM tree
- Implement interactive UI updates through hands-on DOM exercises

DOM Navigation Properties

Every DOM node exposes properties to access its relatives in the tree:

Property	Description	Example
<code>parentNode</code>	Returns the direct parent node	<code>el.parentNode</code>
<code>childNodes</code>	Returns a <code>NodeList</code> of all direct children	<code>el.childNodes[0]</code>
<code>firstChild</code> / <code>lastChild</code>	Returns the first/last child node	<code>el.firstChild</code>
<code>nextSibling</code> / <code>previousSibling</code>	Returns adjacent sibling nodes	<code>el.nextSibling</code>

Important Notes:

- `childNodes` includes **text nodes** (whitespace counts as a node). To skip them, use `children` or check `nodeType === 1`.
- `nodeValue` extracts the text content of a text node.
- `nodeName` returns the tag name in **UPPERCASE** (e.g., `DIV`, `P`).

```
1 // Example: Accessing text inside <h3 id="heading1">Hello</h3>
2 let h3 = document.getElementById("heading1");
3 console.log(h3.firstChild.nodeValue); // "Hello"
4 console.log(h3.nodeName);             // "H3"
```

JS

Creating & Adding Elements

- Create Element:** `document.createElement("tag")`
- Create Text Node:** `document.createTextNode("text")`
- Attach Text to Element:** `element.appendChild(textNode)`
- Insert into DOM:**
 - `parent.appendChild(newEl)` → Adds as **last child**
 - `parent.insertBefore(newEl, referenceEl)` → Inserts **before** a specific child

Removing Elements

Method	Usage	Behavior
<code>element.remove()</code>	Modern, direct	Removes the element itself
<code>parent.removeChild(child)</code>	Legacy	Removes a specific child from its parent

Question 1

Problem Statement

■ Write in lab record

Which function on the document object should be used to retrieve a HTML element uniquely?

Answer

■ Write in lab record

```
1 document.getElementById(id)
```

JS

Note: `document.querySelector("#id")` also works, but `getElementById` is faster and specifically designed for unique ID retrieval.

Question 2

Problem Statement

■ Write in lab record

Which JavaScript function can be used to dynamically add a event listener to a HTML element?

Answer

■ Write in lab record

```
1 element.addEventListener(eventType, callbackFunction)
```

JS

OR

```
1 parentElement.insertBefore(newElement, referenceElement)
```

JS

Question 3

Problem Statement

■ Write in lab record

Which method should be used to add a new HTML child element?

Answer

■ Write in lab record

```
1 parentElement.appendChild(newElement)
```

JS

OR

```
1 parentElement.insertBefore(newElement, referenceElement)
```

JS

Question 4

Problem Statement

■ Write in lab record

Use the DOM API, create a product page which shows the prices of a laptop and a cell phone. You may use paragraph tag to wrap these elements. When the user clicks on add button, you should be able to add TV with price details and when you click on delete button then cell phone details will be deleted.

Preview (Working)

✕ Do not copy. Read for understanding and the viva

Product Details

 Laptop: \$1,200

 Cell Phone: \$800

Add TV

Delete Cell Phone

Code

■ Write in lab record

product-page.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <title>Session 6: DOM Navigation & Manipulation</title>
6     <style>
7       body {
8         font-family: sans-serif;
9         padding: 20px;
10      }
11      .container {
12        max-width: 600px;
13        margin: auto;
14      }
15      p {
16        background: #f0f0f0;
17        padding: 10px;
18        margin: 8px 0;
19        border-radius: 4px;
20      }
21      button {
22        padding: 8px 15px;
23        margin-right: 10px;
24        cursor: pointer;
25      }
26    </style>
27  </head>
28  <body>
29    <div class="container">
30      <h2>Product Details</h2>
31      <div id="productList">
32        <p id="laptop">🖥 Laptop: $1,200</p>
33        <p id="cellphone">📱 Cell Phone: $800</p>
34      </div>
35      <br />
36      <button id="addBtn">Add TV</button>
37      <button id="deleteBtn">Delete Cell Phone</button>
38    </div>
39
40    <script>
41      // Add TV using DOM Navigation & Creation
42      document.getElementById("addBtn").addEventListener("click", () =
43        const tvEl = document.createElement("p");
```

```
44         const tvText = document.createTextNode("📺 TV: $1,500");
45         tvEl.appendChild(tvText);
46
47         // Insert before the delete button's container or at end
48         document.getElementById("productList").appendChild(tvEl);
49     });
50
51     // Delete Cell Phone using removeChild
52     document
53         .getElementById("deleteBtn")
54         .addEventListener("click", () => {
55             const phone = document.getElementById("cellphone");
56             if (phone && phone.parentNode) {
57                 phone.parentNode.removeChild(phone);
58             }
59         });
60     </script>
61 </body>
62 </html>
```

[✎ Edit this page](#)

[< Previous](#)
Session 5

Session 7 [Next >](#)